

Group members



Ahammad, Jony



Kumara, Muditha



Podkorytov, Nikolai



Shrestha Bata, Prabesh

Your answer NOT SUBMITTED

Assignment

Part 1: Simple MLP

Overview

In this assignment, you will build on the foundational neural network concepts learned in the first section. You'll apply these ideas to create and train a simple Multi-Layer Perceptron (MLP) model for your ongoing group project. The goals are:

1. **Understand and implement the forward pass** to generate predictions.
2. **Configure and compute cost (loss) functions** for evaluating model performance.
3. **Demonstrate backpropagation** with gradient updates to optimize the model.

By the end of this activity, you should have an **end-to-end training pipeline** showcasing how data flows from input to output, how errors are quantified, and how weights are updated in response to those errors.

Learning Path and Steps

Step 1: Select a Dataset

- **Task:** Work in your existing project groups.
- **Dataset:** Choose the dataset you've been exploring in previous sessions (e.g., a classification dataset like MNIST or a custom dataset relevant to your project).

- **Preprocessing:**
 - Normalize/standardize your data (e.g., using scikit-learn's [StandardScaler](#)).
 - Split into training and validation sets to monitor performance.

Step 2: Define the Model Architecture

- **Goal:** Construct a simple MLP using a deep learning framework (e.g., TensorFlow Keras).
- **Requirements:**
 - Input layer matching the number of features in your dataset.
 - At least one hidden layer (e.g., 64–128 neurons) with a non-linear activation (like ReLU).
 - Output layer suited to your task (e.g., a single neuron with linear activation for regression, or multiple neurons with softmax for multi-class classification).

Recommended Reading: [Keras Sequential Model Documentation](#)

Step 3: Implement the Forward Pass

- **Objective:** Show how data moves through each layer to produce a prediction.
- **Action Items:**
 1. Create placeholder or random input data for testing.
 2. Feed the data into the model and observe the output predictions.
 3. Inspect layer outputs (if desired) to see how data transforms across layers.

Tips:

- Use `model.predict()` in Keras to get predictions for sample inputs.
- Compare the raw output (logits) vs. activated output (e.g., softmax probabilities).

Step 4: Configure and Compute the Cost Function

- **Purpose:** Quantify the error between predictions and ground truth.
- **Examples:**
 - For classification, use categorical cross-entropy loss.
 - For regression, consider mean squared error.
- **Action Item:**
 - Compile the model with the chosen optimizer (adam or sgd) and loss function.
 - Evaluate the loss by calling `model.evaluate()` or by writing a custom step with a gradient tape (TensorFlow).

Recommended Reading: [Keras Loss Functions](#)

Step 5: Demonstrate Backpropagation

- **Objective:** Show how gradients are computed and used to update weights.
- **Approach:**
 1. Use Keras' built-in training (`model.fit()`) to automatically handle backprop.
 2. Alternatively, demonstrate a manual gradient computation with `tf.GradientTape` for deeper insight.

Reference: [TensorFlow GradientTape Guide](#)

Step 6: Implement the Full Training Loop

- **Task:** Combine your data pipeline, model definition, forward pass, loss function, and optimizer into a loop that runs over multiple epochs.
- **Key Steps:**
 1. Load training data in batches.
 2. Perform forward pass and compute loss.
 3. Compute gradients and update weights.
 4. Track metrics (accuracy, MSE, etc.) over epochs.

Example:

```
history = model.fit(train_data, train_labels, epochs=10,  
validation_data=(val_data, val_labels)) print("Training completed!")  
(Replace with your dataset and labels.)
```

Step 7: Collaborative Implementation and Debugging

- **Collaboration:** Work together, dividing tasks such as data preprocessing, code structure, and debugging.
- **Debugging:** Compare outputs at each step to expected values (e.g., if the loss is not decreasing, investigate learning rate or data shape).
- **Record-Keeping:** Maintain a shared document (e.g., Google Docs or GitHub wiki) to note down experiments, parameter tweaks, and results.

Reference: [TensorFlow Models on GitHub](#) for community examples.

Step 8: Final Deliverable

- **What to Submit:**
 1. **Code** in a Git repository (optional) or attached demonstrating the full pipeline.
 2. **Project Report** summarizing your model architecture, training process, final results, and any challenges encountered.
- **Optional:** Include **visualizations** of training curves (loss and accuracy) per epoch. Tools like [TensorBoard](#) or [Matplotlib](#) can help.
- Include individual reflections.

Part 2: Working with CNN and RNN

Context & Purpose

In the **first part** of this assignment, you built and trained a Multi-Layer Perceptron (MLP). Now, in **Part 2**, you will explore Convolutional Neural Networks (CNNs) or Recurrent

Neural Networks (RNNs) to address more complex data types (image-based or sequential). The knowledge gained from your MLP implementation will guide you in structuring data pipelines, configuring losses, and troubleshooting training issues for these specialized models.

Learning Path & Detailed Steps

Step 1: Choose CNN or RNN Approach

1. CNN (Image-Based Tasks):

- Ideal if your project involves image classification, object detection, or visual data.
- Build upon your dataset by reshaping/normalizing images.
- Use convolution and pooling layers to extract spatial features.

2. RNN (Sequential Data):

- Suitable for text classification, sentiment analysis, time-series forecasting, etc.
- Design an RNN (LSTM or GRU) to capture dependencies across time steps or sequence positions.

Tip: Select whichever architecture aligns best with your existing project dataset and goals.

Step 2: Adapt Your Dataset

1. Data Preparation:

- If you're using CNNs, ensure your images are loaded with a consistent shape (e.g., 28×28 or 32×32 etc.) and normalized/pixel-scaled.
- For RNN tasks, transform sequences into a consistent length using padding or truncation.

2. Existing Project Data:

- Integrate these transformations into your pipeline from Part 1, preserving train/validation splits.

Step 3: Define the Model

1. CNN:

- Start with one or two convolution layers (e.g., 32 or 64 filters) followed by a pooling layer.
- Add one or more fully connected layers for classification.

2. RNN:

- Use an Embedding layer (for text input) or feed in numeric sequences directly.
- Add an LSTM or GRU layer with a suitable hidden size (e.g., 64–128 units).
- Optionally stack multiple RNN layers for more complex patterns.

Step 4: Forward Pass & Prediction

1. Single Sample Test:

- Use `model.predict()` on a single image or a single sequence.
- Verify output shapes and preliminary predictions.

2. Interpretation:

- For CNN, visualize intermediate feature maps (optional) to see what filters detect.
- For RNN, consider how hidden states evolve over time steps (advanced).

Step 5: Loss & Metrics

1. Loss Functions:

- CNN classification: Typically categorical cross-entropy.
- RNN classification: Same approach if it's a classification task (e.g., sentiment or multi-class).
- If you're doing a regression approach (e.g., predicting future values in a time series), use mean squared error or mean absolute error.

2. Key Metrics:

- Classification: Accuracy, possibly F1-score or confusion matrix analysis.
- Regression: Mean squared error, R^2 , or other domain-specific metrics.

Step 6: Training & Debugging

1. Train:

- Use `model.fit()` with suitable batch size and epoch count (e.g., 10–20).
- Track validation loss/accuracy each epoch.

2. Debug:

- Check for issues like rapidly increasing loss (potential exploding gradients) or no improvement over epochs (learning rate might be too low/high).
- Print shapes of your input data or intermediate outputs if dimension mismatches arise.

3. Hyperparameter Tuning:

- Adjust filter/hidden size, number of layers, and learning rate in iterative tests.
- Document each experiment in a table/log.

Step 7: Evaluate Overfitting & Underfitting

1. Overfitting:

- If validation accuracy is significantly lower than training accuracy, employ regularization or data augmentation.

2. Underfitting:

- Increase model capacity or train longer if both training and validation accuracy are low.

Note: Plot accuracy/loss curves to see if the model saturates or diverges.

Step 8: Final Results & Reflection

1. Reporting:

- Summarize your final architecture, hyperparameters, and performance metrics.
- Show any confusion matrices, classification reports, or sample predictions as relevant.

2. Comparison:

- If you tested both CNN and RNN, or different hyperparameter settings, briefly discuss how they compare.

3. Project Integration:

- Explain how these insights help refine or extend your overall group project.

- Could these models be combined with your MLP approach for ensemble methods?

Deliverables

1. **Code Submission:** Jupyter Notebook / .py files showing your CNN/RNN architecture and training pipeline. Include screenshots of outputs.

2. **Short Reflection:**

- A few paragraphs describing challenges faced, how you addressed them, and any surprising results. Individually for each member.
- Clarify next steps or open questions for further exploration.

3. **(Optional) Visualizations:**

- Plots of training/validation loss & accuracy.
- If you implemented advanced visualization (e.g., feature maps in CNN, hidden state evolution in RNN), include screenshots or discussion.

The deadline has passed. You can still submit your answer, but it will be marked as submitted too late.